

Computational and Statistical Pipeline for Proteomic Causal Discovery

Proteomics Pipeline Formalization

July 11, 2026

1 Introduction

This document formalizes the computational workflow for causal discovery in proteomic datasets derived from mass spectrometry. We treat the system as a directed acyclic graph (DAG) where experimental perturbations are utilized to minimize structural entropy.

2 Formal Pipeline Definition

The pipeline is defined as a sequence of morphisms acting upon the data.

2.1 Preprocessing Morphism (Φ)

Raw intensity matrices $\mathbf{D}$ are variance-stabilized via log-transformation and normalization:

$$\Phi(\mathbf{D}) \rightarrow \mathbf{D}^* \quad (1)$$

- **Analysis Tool:** `limma` or `vsn` in R.
- **Key Paper:** *Huber et al. (2002)* on Variance Stabilization for microarray/proteomics data.
- **GPU Optimization:** Use `torch` for R to perform matrix operations directly in VRAM.

2.2 Topology Inference (Ψ)

We infer the network $\mathcal{G}$ by maximizing the posterior probability given constraints $\mathcal{K}$:

$$\Psi(\mathbf{D}_{\text{sub}}, \mathcal{K}) = \operatorname{argmax}_{G \in \mathcal{G}} [P(\mathbf{D}_{\text{sub}}|G) \cdot P(G|\mathcal{K})] \quad (2)$$

- **Analysis Tool:** `bnlearn` for structural discovery.
- **Key Paper:** *Pearl, J. (2000)* Causality: Models, Reasoning, and Inference.
- **GPU Optimization:** Parallelize structure learning permutations using `foreach` with `doParallel`.

3 Optimal Growth and Constraint Formalization

To ensure the model increases its validity, we prioritize experiments that resolve the highest causal ambiguity (entropy).

3.1 Active Learning: Expected Information Gain (EIG)

We select the next experimental intervention v^* to maximize EIG:

$$v^* = \operatorname{argmax}_{v_i \in V} [\mathbb{H}(\mathcal{G}_t) - \mathbb{E}_{\mathcal{O}}[\mathbb{H}(\mathcal{G}_t | \mathcal{O}_{\text{exp}})]] \quad (3)$$

- **Analysis Tool:** `ActiveLearning` R packages; custom Shannon entropy functions.
- **GPU Optimization:** Pre-calculate the entropy of all potential interventions (*do*-calculus) simultaneously on a GPU.

3.2 Complexity Constraints (MDL Principle)

We constrain the size of $\mathcal{G}$ to prevent overfitting, using the Minimum Description Length principle:

$$\text{Cost}(\mathcal{G}) = -\log P(\mathbf{D}_{\text{sub}} | \mathcal{G}) + \frac{\lambda}{2} \log(n) \cdot |\mathcal{G}| \quad (4)$$

- **Analysis Tool:** `glmnet` or `lavaan` for structural sparsity.
- **Key Paper:** *Rissanen, J. (1978)* Modeling by shortest data description.

4 Summary of Analysis Pipeline

The following table summarizes the R-based tools recommended for each pipeline stage.

Stage	R Tool	Purpose
Preprocessing	<code>limma</code>	Normalization/Batch effect
Reduction	<code>Boruta</code>	Feature selection
Inference	<code>bnlearn</code>	DAG structure learning
Validation	<code>caret</code>	Predictive performance

Table 1: Recommended R-based pipeline tools.